

Final Exam : Compléments d'informatique - Correction

ENSAI 2A UE3

2026-09-28

Instructions

- Written exam
- Duration: 2 hours
- One A4 sheet of **handwritten** notes (front and back) allowed
- No calculator

! Read Before Starting

- You can answer the exercises in any order.
- Within each exercise, the questions are not necessarily ordered by increasing difficulty.
- Unless otherwise stated, the questions expect short answers.
- Strictly follow the instructions. If a question seems unclear, state your assumptions on your answer sheet.
- When code is required, follow the basic language conventions (keywords, indentation, etc.). Syntax errors due to handwriting will be tolerated. If you do not remember the exact name of a Python method, you may use pseudocode.
- Unless explicitly requested, code documentation is not required.

Good Luck!

1 Short course questions (3 points)

a. Give six examples of files or folders commonly found in an IT project.

For example:

- README.md
- src/
- tests/
- .gitignore
- pyproject.toml
- .env

b. List the main actors involved in an IT project.

- Client
- Product owner
- Project manager
- Functional team
- Technical team
- Users

c. What is a design pattern?

A design pattern is a characteristic arrangement of modules, recognized as a best practice in response to a software design problem.

d. Which HTTP methods correspond to the CRUD operations?

- POST: Create
- GET: Read
- PUT: Update
- DELETE: Delete

e. Write an HTTP request needed to create a new Pokemon (name: Ponyta, type: Fire, level: 12) in <https://pokedex-api> (choose an appropriate endpoint).

```
POST https://pokedex-api/pokemon
```

```
Body:
```

```
{
  "name": "Ponyta",
  "type": "Fire",
  "level": 12
}
```

2 Multiple Choice Questions (8 points)

2.1 Development tools: which statements are correct?

- a. `pytest` is used to run unit tests. ☒
- b. `psycopg2` is used to communicate with PostgreSQL. ☒
- c. `requests` is used to communicate with PostgreSQL. ☐
- d. `FastAPI` is mainly used to format Python source code. ☐
- e. The `abc` module can be used to define abstract classes. ☒
- f. Swagger can be used to explore and test an API. ☒

2.2 Why can logging be preferable to using `print()` statements when debugging an application?

- a. Logs can indicate the severity of a message. ☒
- b. Logs can be configured to be written to a file. ☒
- c. Logs can provide information about when an event occurred. ☒
- d. `print()` statements cannot display any information about an error. ☐
- e. Logging is only useful when an application crashes. ☐

2.3 Which statements describe good unit testing practices?

- a. A unit test should verify a specific behaviour. ☒
- b. Tests should be reproducible. ☒
- c. A test should ideally be independent of unrelated external systems. ☒
- d. A unit test must always access the real production database. ☐
- e. Tests can help detect regressions after modifying the code. ☒
- f. A unit test depends on a specific execution order. ☐

2.4 Which statements about mocks are correct?

- a. A mock always tests the real implementation of the component it replaces. ☐
- b. It can be used to isolate the code being tested from its external dependencies. ☒
- c. A mock allows a test to simulate specific behaviors of a dependency. ☒
- d. It is only useful when testing code that accesses a database. ☐
- e. A mock is a simulated object that replaces a real component during a test. ☒

2.5 A user submits a form to create a new player. Which sequence best represents the flow of a request through the application layers?

- a. DAO → Controller → Service → Database ☐
- b. Controller → DAO → Service → Database ☐
- c. Service → Client → DAO → Database ☐
- d. Controller → Service → DAO → Database ☒
- e. Service → Controller → DAO → Database ☐

2.6 Which statements describe advantages of a well-designed layered application?

- a. Controllers should contain most of the business logic. ❌
- b. Each layer has a specific responsibility. ✅
- c. If the database management system (DBMS) changes, the impact can be more limited because database-specific code is mainly isolated in the DAO layer. ✅
- d. Any layer can freely access and modify the database. ❌
- e. Separating responsibilities can make the application easier to read, debug, and maintain. ✅
- f. Business logic is separated from data access and user interaction. ✅

2.7 A user clicks a button in the UI (User interface) to retrieve their games. Which statements are correct?

- a. The database receives the user's click directly from the UI. ❌
- b. The frontend sends an HTTP request to the backend API. ✅
- c. The backend processes the request and asks the database for the required data. ✅
- d. The database can return data to the backend, which then sends a response to the frontend. ✅
- e. The UI must always be implemented in the backend. ❌
- f. The API can be used as an interface between the frontend and the backend. ✅

2.8 Which HTTP requests are appropriate for retrieving all games played by player 96?

- a. `PUT https://my-awesome-url/game?id_player=96` ❌
- b. `GET https://my-awesome-url/game?id_player=96` ✅
- c. `PUT https://my-awesome-url/game/96` ❌
- d. `GET https://my-awesome-url/player/96` ❌
- e. `GET https://my-awesome-url/player/69/games` ❌
- f. `GET https://my-awesome-url/player/96/games` ✅

2.9 Which statements are correct about environment variables?

- a. Environment variables can be used to configure the database connection. ✅
- b. A password can be stored in an environment variable. ✅
- c. The `.env` file containing secrets should not be committed to the remote repository. ✅
- d. Database credentials should preferably be hard-coded in every DAO. ❌
- e. Using configuration variables makes it easier to change environments. ✅
- f. Using environment variables means that secrets cannot be accessed by the application. ❌

3 Code (5 points)

Below are two classes representing business objects in a gaming application.

```
from business_object.player import Player

class Game:
    """Business object representing a Game between two players."""
    def __init__(self, player1: Player, player2: Player, game_mode: str,
                  winner: Player | None, id_game: int=None):
        self.id_game = id_game
        self.player1 = player1
        self.player2 = player2
        self.game_mode = game_mode
        self.winner = winner
```

```
class Player:
    """Business object representing a Player."""
    def __init__(self, username: str, elo: int, pokemon_fan:bool=False, id_player=None):
        self.id_player = id_player
        self.username = username
        self.elo = elo
        self.pokemon_fan = pokemon_fan
```

a. Create a coin flip Game between Fred and Jamy.

```
fred = Player("Fred", 1000)
jamy = Player("Jamy", 1000)

game = Game(
    player1=fred,
    player2=jamy,
    game_mode="coinflip",
    winner=jamy)
```

b. Create a new method that returns a string representation of the Game, such as:

- “coinflip between Fred and Jamy. Winner: Jamy”

```
def __str__(self):
    winner = self.winner.username if self.winner else "None"
    return f"{self.game_mode} between {self.player1.username} and {self.player2.username}. Winner: {winner}"
```

c. Write an SQL query to list all dice games won by the first player (*player1*) ordered by the game's primary key.

```
SELECT *
FROM game
```

```
WHERE game_mode = 'dice'
AND id_winner = id_player1
ORDER BY id_game;
```

d. Complete the end of the `find_by_id()` method below.

The two players must first be retrieved using their IDs.

Finally, the database row is converted into a `Game` object.

```
class GameDao(Singleton):
    def find_by_id(self, id_game: int) -> Game | None:
        """
        Retrieves a specific game and converts the database row into a Game object.
        Args:
            id_game (int): The ID of the game to find.
        Returns:
            Game: The game object if found, otherwise None."""
        res = None
        with DBConnection().connection as connection:
            with connection.cursor() as cursor:
                cursor.execute(
                    """
                    SELECT *
                    FROM game
                    WHERE id_game = %(id_game)s;
                    """,
                    {"id_game": id_game},
                )
                res = cursor.fetchone()
        if not res:
            return None
        p1 = PlayerDao().find_by_id(res["id_player1"])

        # ...

        p2 = PlayerDao().find_by_id(res["id_player2"])
        winner = None
        if res["id_winner"] is not None:
            winner = PlayerDao().find_by_id(res["id_winner"])

        return Game(
            player1=p1,
            player2=p2,
            game_mode=res["game_mode"],
            winner=winner,
            id_game=res["id_game"]
        )
```

e. Following the documentation, write the code for this method.

```
class GameModeFactory:
    @classmethod
    def get_mode(cls, game_mode: str) -> GameMode:
        """
        Returns the corresponding GameMode object.
        Args:
            game_mode (str): The identifier of the game mode (e.g., 'coinflip', 'dice').
        Returns:
            CoinFlipMode or DiceMode: An instance of a class implementing GameMode.
        Raises:
            ValueError: If the requested game_mode is not supported."""
        if game_mode == "coinflip":
            return CoinFlipMode()
        elif game_mode == "dice":
            return DiceMode()
        else:
            raise ValueError(f"Unknown game mode: {game_mode}")
```

f. Complete this test, and write two more to test the method `get_mode()`.

```
def test_unknown_mode():
    with pytest.raises(ValueError):
        GameModeFactory.get_mode("unknown")

# ...

def test_coinflip_mode():
    # GIVEN
    game_mode = "coinflip"

    # WHEN
    mode = GameModeFactory.get_mode(game_mode)

    # THEN
    assert isinstance(mode, CoinFlipMode)

def test_dice_mode():
    # GIVEN
    game_mode = "dice"

    # WHEN
    mode = GameModeFactory.get_mode(game_mode)

    # THEN
    assert isinstance(mode, DiceMode)
```

4 Git (4 points)

Sofia has just joined your development team as a junior developer.

a. **Which Git command should she use to retrieve a local copy of the repository?**

She should clone the repository:

```
git clone https://github.com/goodfood/ten-vegetables-per-day.git

# If she use a token
git clone https://$TOKEN@github.com/goodfood/ten-vegetables-per-day.git
```

b. **What is the purpose of using a personal access token when authenticating to GitHub over HTTPS?**

A personal access token is used to authenticate the user when communicating with GitHub over HTTPS.

c. **Describe the Git workflow you use each week to update the shared report project file.**

If you are not working on the project: please state this and imagine that each week you have to fill in a file containing a task list and send it to the shared repository.

A possible workflow is:

```
git pull
# I then modify the report file.
git status
git add .
git commit -m "Update weekly report"
git push
```

Sofia modifies *src/vegetables.py* and tries to push her changes, but Git says “Everything up-to-date” / “Nothing to push”.

d. **What does she need to do before pushing her changes?**

She has modified a file, so she needs to add and commit the changes first:

```
git add src/vegetables.py # or git add .
git commit -m "Update vegetables"
```

Then she should be able to push.

Sofia tries to push her commit again, but this time Git displays a different message.

```
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/goodfood/ten-vegetables-per-day.git'
hint: Updates were rejected because the remote contains work that you do
```

hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes.

e. Why is Sofia getting this message? What should she do to resolve the situation?

The remote repository contains commits that are not present in her local repository.

For example, another developer may have pushed changes since Sofia's last `git pull`.

She should first retrieve the remote changes using `git pull`.

If there is no conflict, she can then push her changes.

If a merge conflict occurs, she must resolve the conflict manually, then add and commit the resolution before pushing again.

f. Is the project necessarily lost? Explain why.

No.

Stan still has a local copy of the Git repository. A local Git repository contains the project's files and its Git history. The project can therefore be recovered from Stan's local repository.